Ain Shams University – Faculty of Engineering

Computer & Systems Engineering Department

CSE 335s: Operating Systems

Memory Management Assignment

Memory Allocation using Segmentation

Zeyad Saied Zakria
2300248
Sec: 2

Github Link: ZeyadXP/memory-allocation
exe link:
https://drive.google.com/drive/folders/1Oe61khyXCej7NKcni7WXOG_PV_sxpjbg?usp=drive_link

the .exe file is also in the dist folder on github

1. Introduction

Memory management is one of the most fundamental responsibilities of an operating system. This project implements a Memory Allocation Simulator based on the Segmentation technique, in which a process is divided into variable-length logical segments (Code, Data, Stack, etc.) that are placed independently into free memory holes.

Two classical dynamic-allocation algorithms are supported:

- First-Fit: allocates a segment into the first hole whose size ≥ the requested size, scanning from the lowest address.
- Best-Fit: allocates a segment into the smallest hole that still accommodates it, minimising leftover wasted space.

The project delivers a Python desktop application with a live graphical interface, plus this report containing code screenshots and full memory-layout snapshots after every operation.

2. Data Structures & Code Explanation

2.1 Core Classes

Three Python classes model the system:

Core Classes: Segment, Process, MemoryManager

```
class Segment:
    def __init__(self, name, size, start=None):
        self.name  = name    # segment name (e.g. 'Code')
        self.size  = size    # size in K
        self.start = start   # base address (None if unallocated)

class Process:
    def __init__(self, pid):
        self.pid      = pid
        self.segments = []   # list of Segment objects

class MemoryManager:
    def __init__(self, total_size):
        self.total     = total_size
        self.holes     = []  # [{start, size}, ...]  sorted
        self.allocated = []  # [{start,size,pid,seg_name}, ...]
        self.processes = {}  # {pid -> Process}
```

Figure 1 – Core classes: Segment, Process, MemoryManager

The MemoryManager holds two key tables updated after every operation:

- Free Partitions Table (holes): a list of {start, size} dicts, kept sorted by start address.
- Allocated Partitions Table (allocated): a list of {start, size, pid, seg_name} dicts.

2.2 First-Fit Algorithm

The simplest strategy: scan holes in address order and return the first one large enough.

First-Fit Algorithm

```
def _first_fit(self, size):
    """Return the first hole large enough for 'size'."""
    for h in self.holes:   # holes are sorted by start address
        if h['size'] >= size:
            return h       # found — return immediately
    return None            # no suitable hole found
```

Figure 2 – First-Fit implementation

Time complexity: O(n) where n is the number of free holes. Stops at the first match, making it the fastest of the two algorithms.

2.3 Best-Fit Algorithm

Best-Fit picks the hole that wastes the least space by choosing the smallest eligible hole.

Best-Fit Algorithm

```
def _best_fit(self, size):
    """Return the smallest hole that still fits 'size'."""
    candidates = [h for h in self.holes if h['size'] >= size]
    if not candidates:
        return None
    return min(candidates, key=lambda h: h['size'])
```

Figure 3 – Best-Fit implementation

Time complexity: O(n) — must scan all holes to find the minimum. Produces less external fragmentation per allocation but may leave many tiny unusable holes over time.

2.4 Allocation – Atomic Two-Phase Commit

A dry-run on a deep copy of the holes list verifies that every segment of the process fits before any change is committed. If any segment fails, the original holes list is restored (rollback).

allocate_process — Atomic Allocation

```
def allocate(self, pid, segs_dict, algo):
    fit = self._first_fit if algo == 'first' else self._best_fit
    snap = copy.deepcopy(self.holes)  # snapshot for rollback
    tmp  = []
    for name, size in segs_dict.items():
        h = fit(size)
        if h is None:              # segment does not fit
            self.holes = snap      # rollback all changes
            return False, f'Segment {name} could not fit'
        start = h['start']
        if h['size'] == size:      # exact fit — remove hole
            self.holes.remove(h)
        else:                      # partial fit — shrink hole
            h['start'] += size
            h['size']  -= size
        tmp.append((name, size, start))
    # Commit all segments
    proc = Process(pid)
    for name, size, start in tmp:
        proc.segments.append(Segment(name, size, start))
        self.allocated.append({...})
    self.processes[pid] = proc
    return True, f'Process {pid} allocated successfully.'
```

Figure 4 – allocate_process with rollback

2.5 Deallocation & Hole Merging

When a process is freed, all its segments are returned to the holes list. Adjacent free blocks are then merged (coalesced) to prevent fragmentation from growing over time.

deallocate_process — Free & Merge

```
def deallocate(self, pid):
    proc = self.processes.pop(pid)
    for s in proc.segments:
        # Return segment memory as a free hole
        self.holes.append({'start': s.start, 'size': s.size})
        self.allocated.remove(...)  # remove from allocated list
    self._merge_holes()  # coalesce adjacent free blocks

def _merge_holes(self):
    self.holes.sort(key=lambda h: h['start'])
    merged = []
    for h in self.holes:
        if merged and merged[-1]['start']+merged[-1]['size'] >= h['start']:
            # Adjacent or overlapping → extend previous
            merged[-1]['size'] = max(...) - merged[-1]['start']
        else:
            merged.append(dict(h))
    self.holes = merged
```

Figure 5 – deallocate_process and _merge_holes

3. Test Case Scenario

The following scenario is executed twice — once with First-Fit and once with Best-Fit. After each operation, a snapshot of the memory layout and the segment/holes tables is shown.

Initial Setup

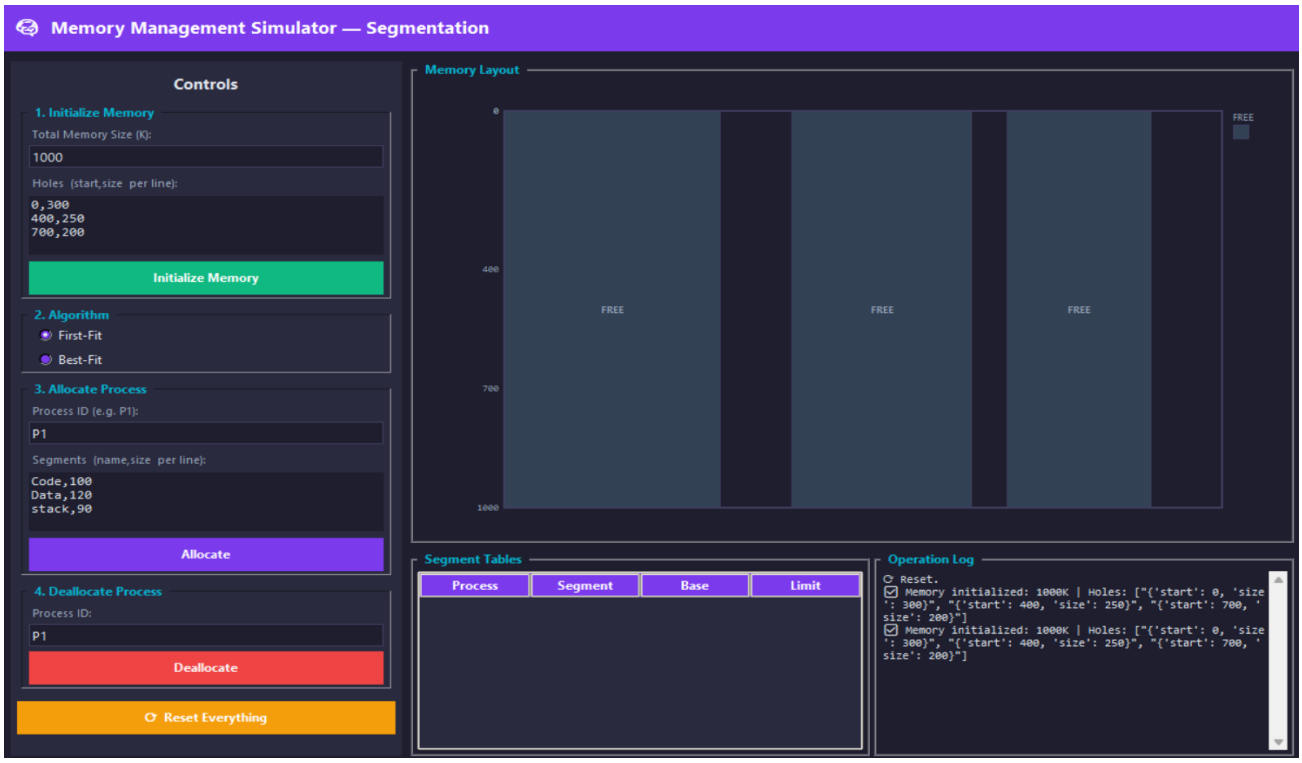
- Total Memory Size: 1000 K
- H1: Start = 0, Size = 300 K
- H2: Start = 400, Size = 250 K
- H3: Start = 700, Size = 200 K
- Regions 300–399 and 650–699 are pre-occupied (OS / fixed use).

Operations in order:

- Allocate P1 — Code=100K, Data=120K, Stack=90K
- Allocate P2 — Code=200K, Data=40K
- Allocate P3 — Code=120K, Data=50K
- Deallocate P1
- Allocate P4 — Code=230K, Data=40K

4. Test Case 1 – First-Fit

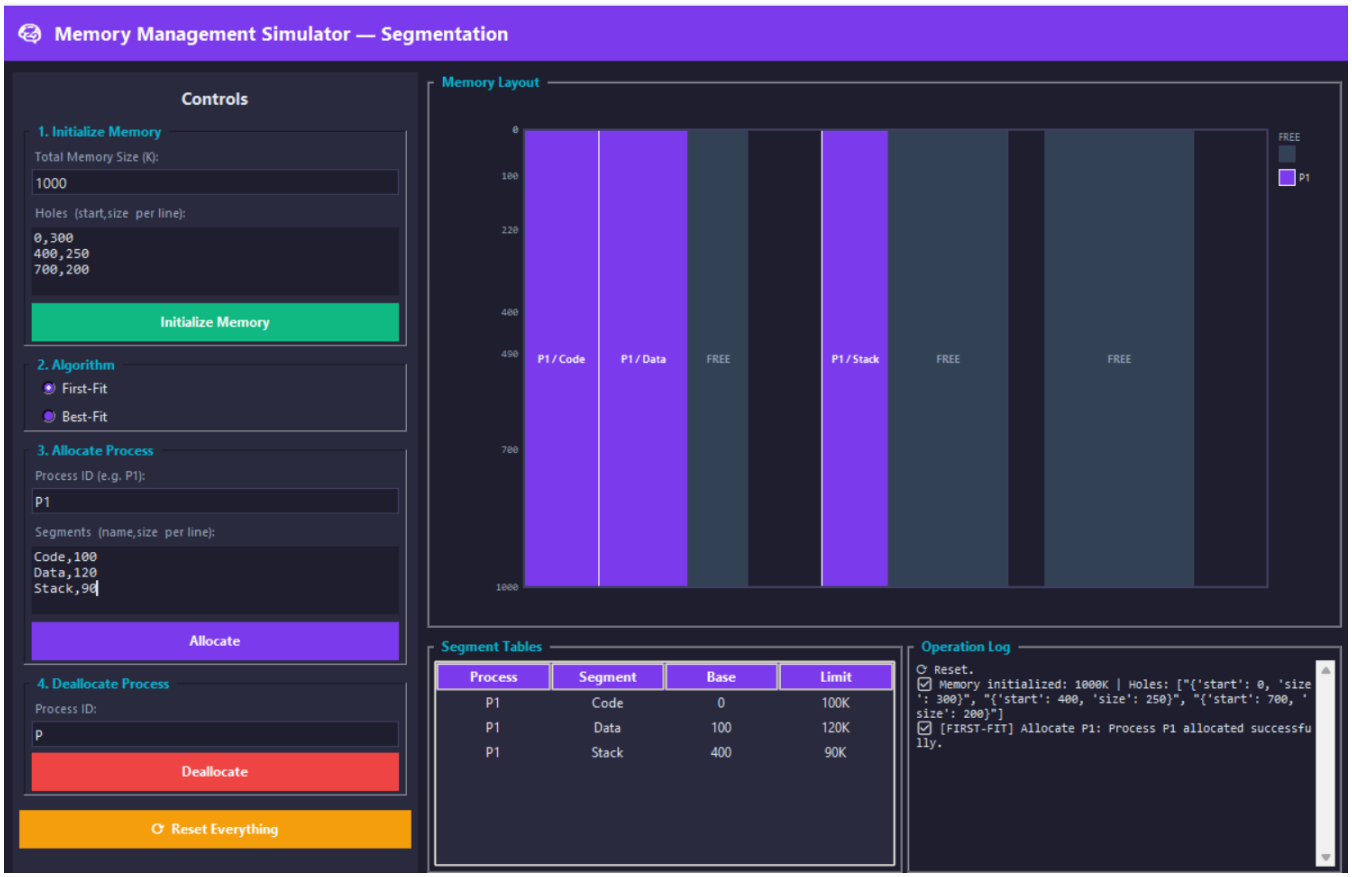
Initial State



Snapshot 1 – Initial memory: three free holes (First-Fit)

Step 1 – Allocate P1 (Code=100K, Data=120K, Stack=90K)

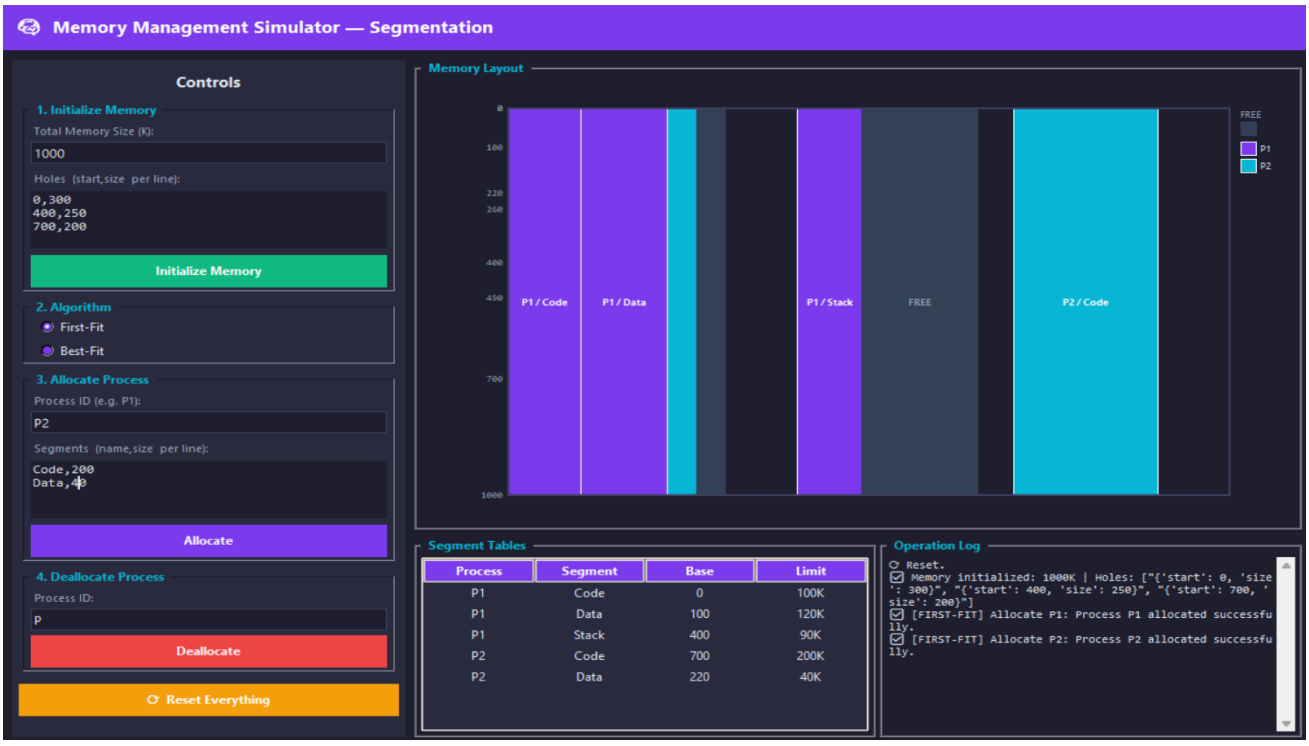
First-Fit scans from address 0. H1 (300K) fits Code (100K) → placed at 0, H1 shrinks to (100, 200K). H1 still fits Data (120K) → placed at 100, H1 shrinks to (220, 80K). H2 (250K) fits Stack (90K) → placed at 400, H2 shrinks to (490, 160K).



Snapshot 2 – After allocating P1 (First-Fit)

Step 2 – Allocate P2 (Code=200K, Data=40K)

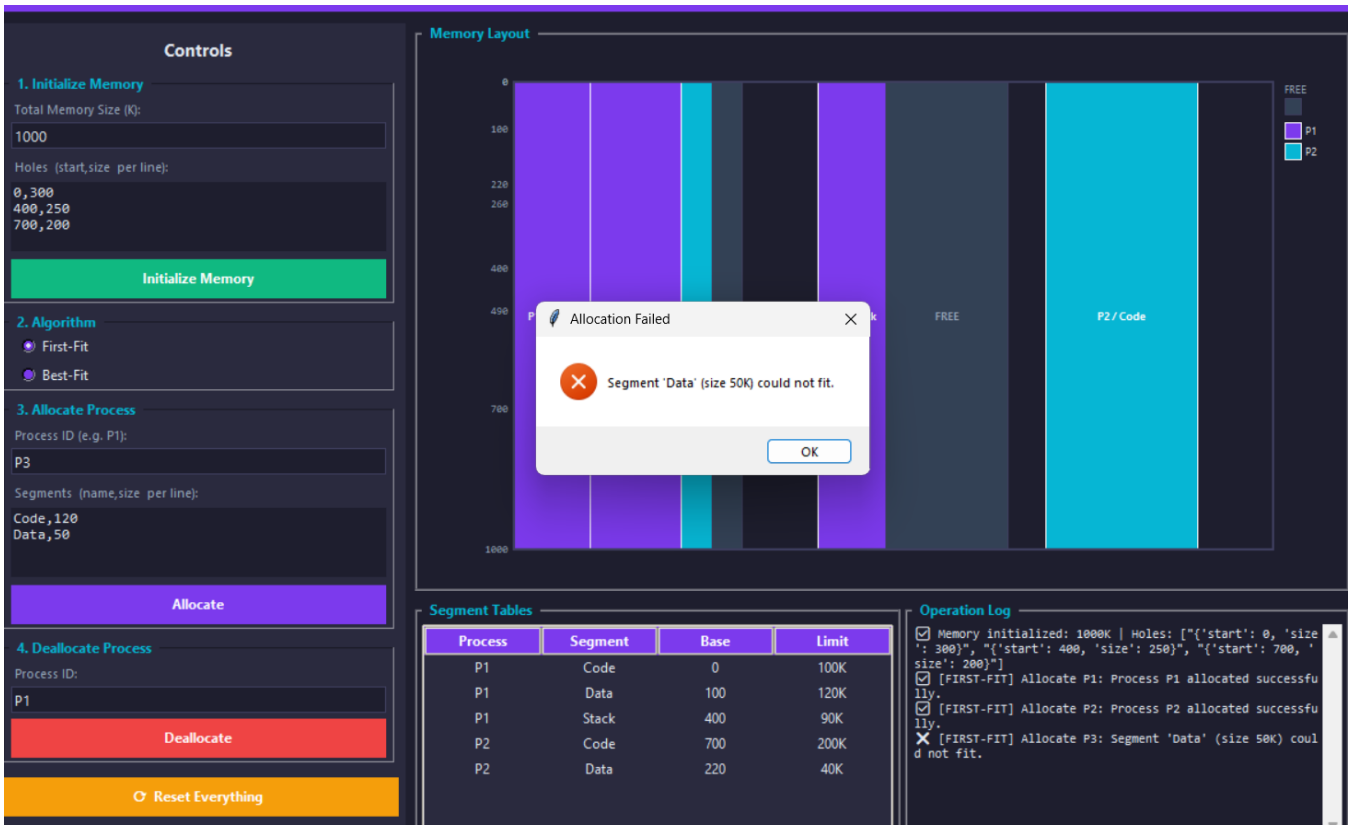
Code (200K): H1-rem (80K) too small, H2-rem (160K) too small, H3 (200K) fits exactly → placed at 700, H3 exhausted. Data (40K): H1-rem (80K) is first fit → placed at 220, H1 shrinks to (260, 40K).



Snapshot 3 – After allocating P2 (First-Fit)

Step 3 – Allocate P3 (Code=120K, Data=50K) ← FAILS

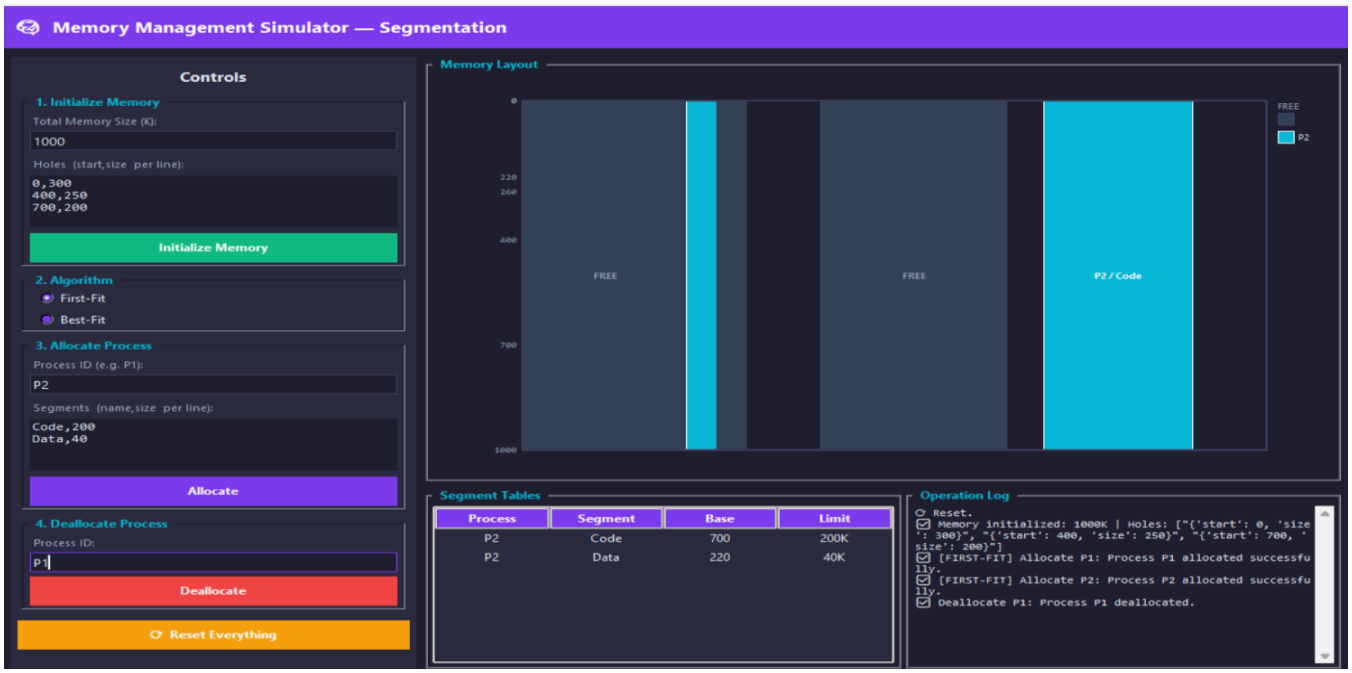
Code (120K) would fit H2-rem (160K), but then Data (50K) finds no remaining hole ≥ 50K (only 40K and 40K left). The allocation is rolled back. P3 is rejected.



Snapshot 4 – P3 rejected, memory unchanged (First-Fit)

Step 4 – Deallocate P1

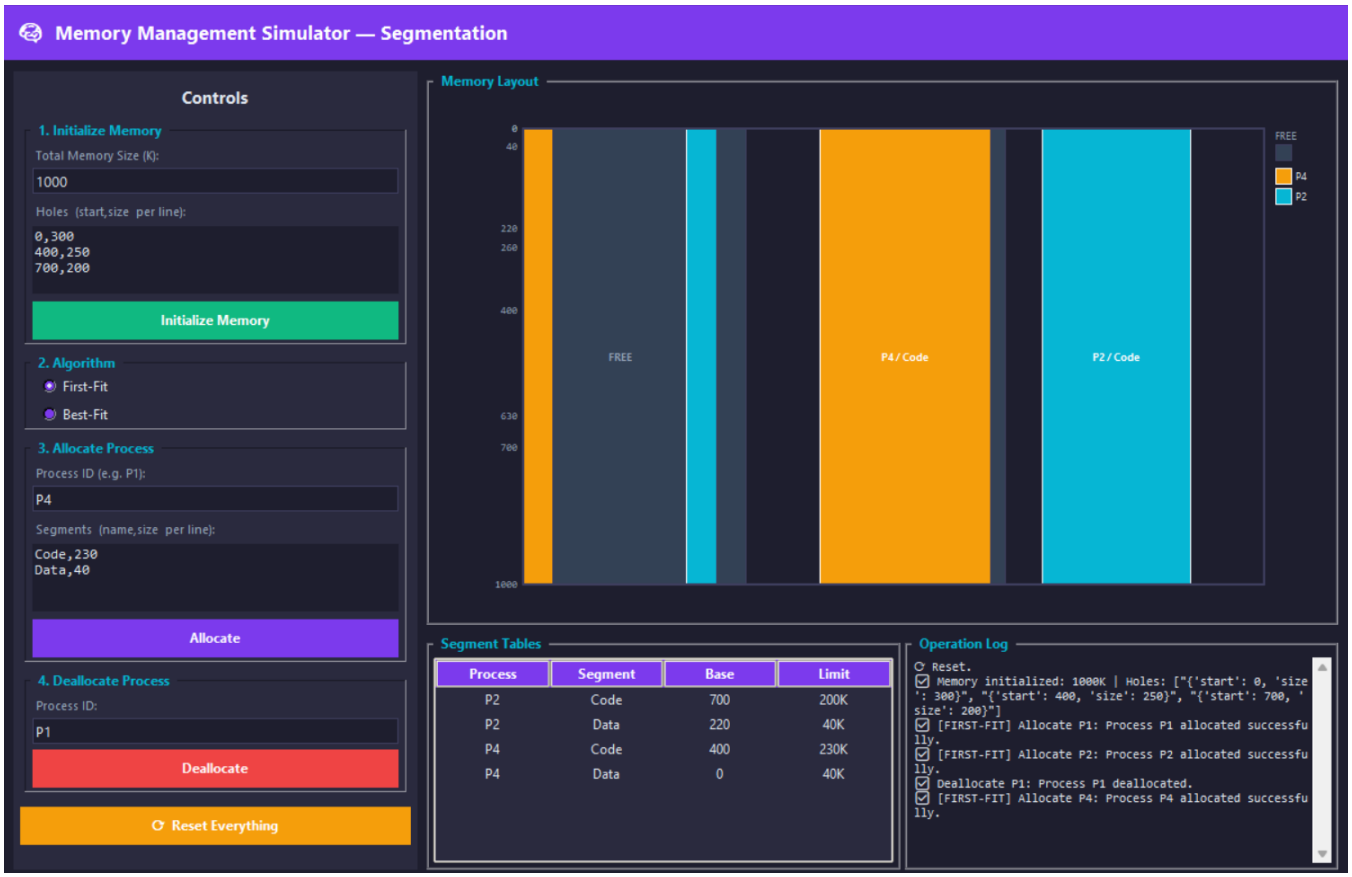
P1's three segments (Code@0/100K, Data@100/120K, Stack@400/90K) are freed. Code and Data are adjacent → merged into (0, 220K). Stack at 400 is adjacent to the former H2-rem at 490 → merged into (400, 250K).



Snapshot 5 – After deallocating P1, holes merged (First-Fit)

Step 5 – Allocate P4 (Code=230K, Data=40K)

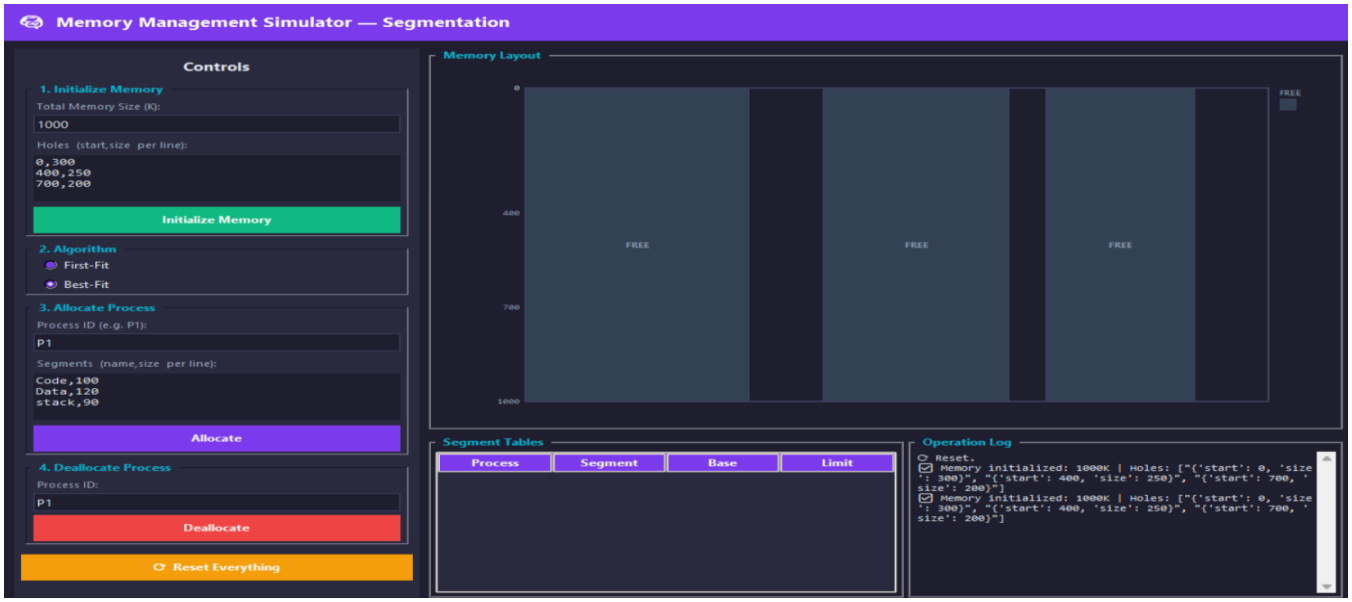
Code (230K): hole (0,220K) too small, hole (260,40K) too small, hole (400,250K) fits → placed at 400, remainder (630, 20K). Data (40K): hole (0,220K) fits → placed at 0, remainder (40, 180K).



Snapshot 6 – After allocating P4 (First-Fit)

5. Test Case 2 – Best-Fit

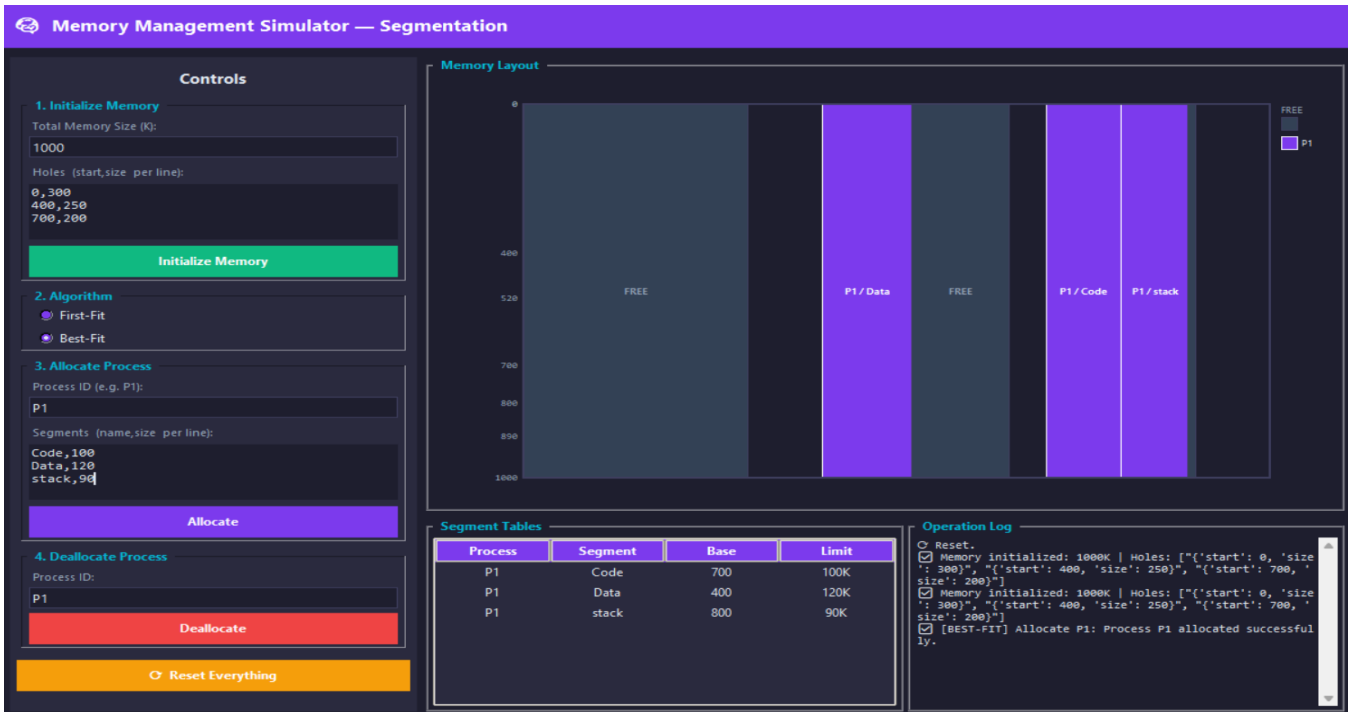
Initial State



Snapshot 7 – Initial memory: three free holes (Best-Fit)

Step 1 – Allocate P1 (Code=100K, Data=120K, Stack=90K)

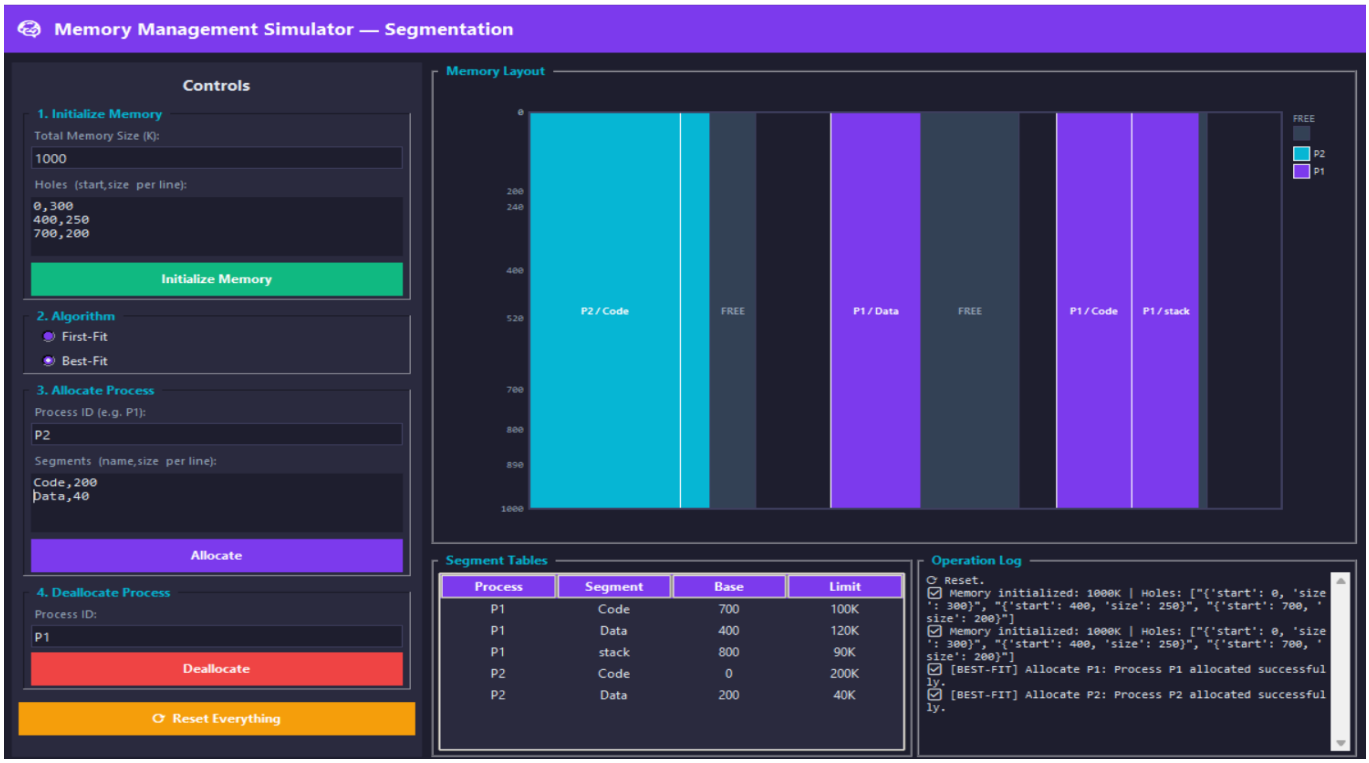
Best-Fit picks the smallest sufficient hole at each step. Code (100K): H3 (200K) is the smallest fit → placed at 700, H3 rem = (800, 100K). Data (120K): H2 (250K) is now smallest fit (H3-rem=100K < 120K) → placed at 400, H2 rem = (520, 130K). Stack (90K): H3-rem (100K) is smallest fit → placed at 800, H3 rem = (890, 10K).



Snapshot 8 – After allocating P1 (Best-Fit)

Step 2 – Allocate P2 (Code=200K, Data=40K)

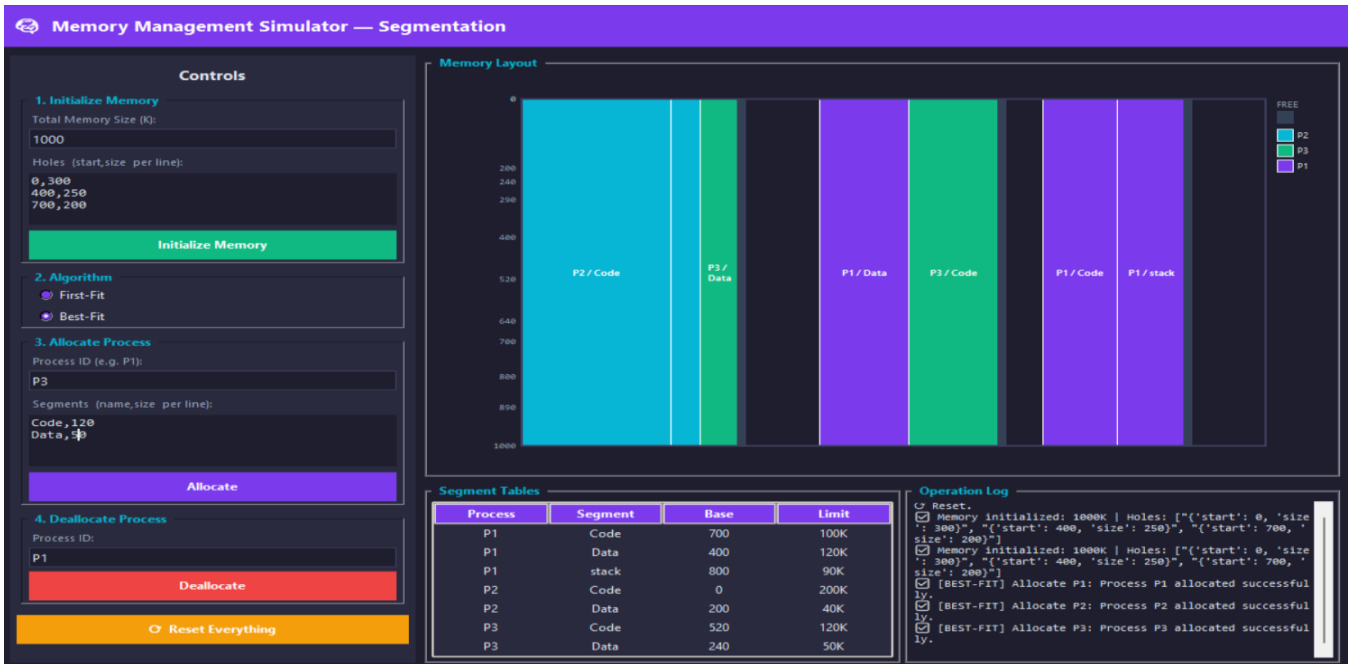
Code (200K): H2-rem (130K) too small; H1 (300K) is the only fit → placed at 0, H1 rem = (200, 100K). Data (40K): H3-rem (10K) too small; H1-rem (100K) is now smallest fit → placed at 200, H1 rem = (240, 60K).



Snapshot 9 – After allocating P2 (Best-Fit)

Step 3 – Allocate P3 (Code=120K, Data=50K) ← SUCCEEDS

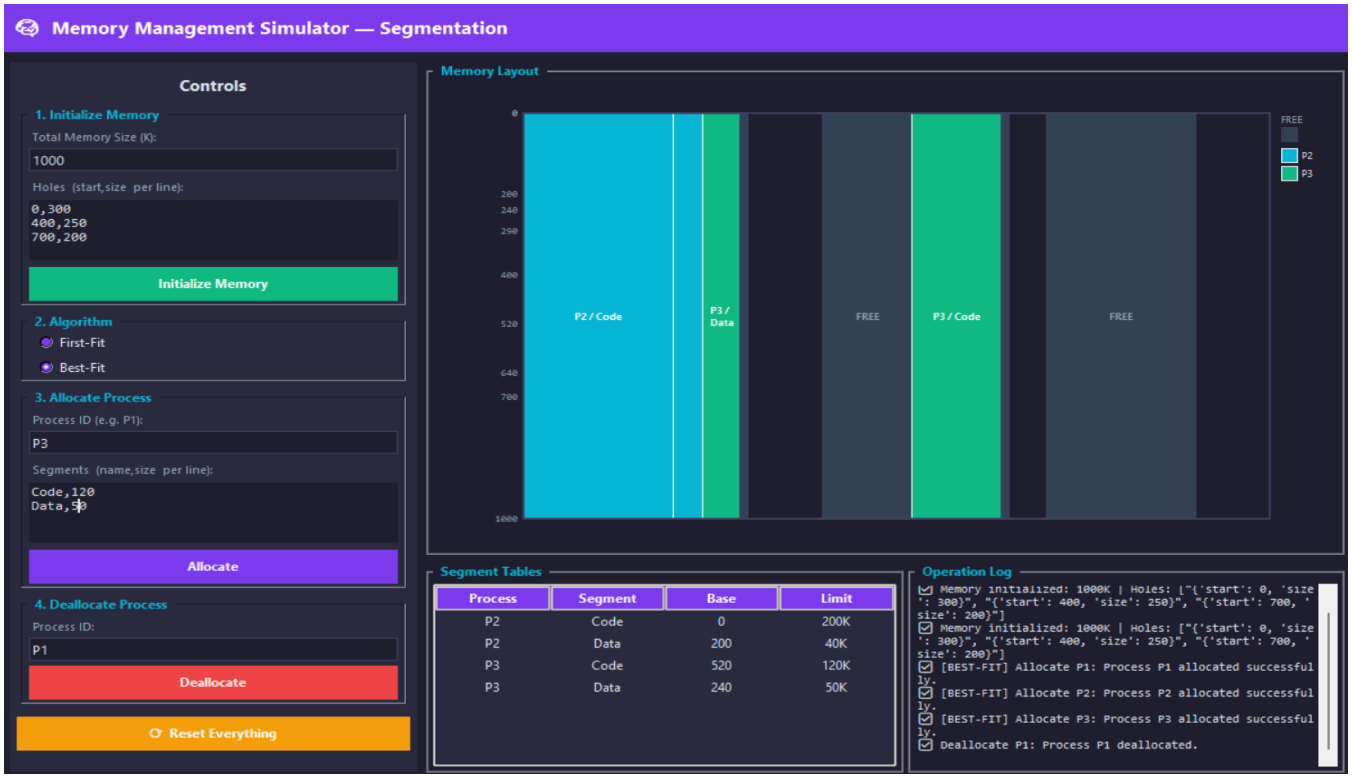
Code (120K): H2-rem (130K) is the smallest fit → placed at 520, H2 rem = (640, 10K). Data (50K): H1-rem (60K) is the only fit → placed at 240, H1 rem = (290, 10K). P3 succeeds under Best-Fit because the earlier allocations conserved larger holes.



Snapshot 10 – After allocating P3 (Best-Fit)

Step 4 – Deallocate P1

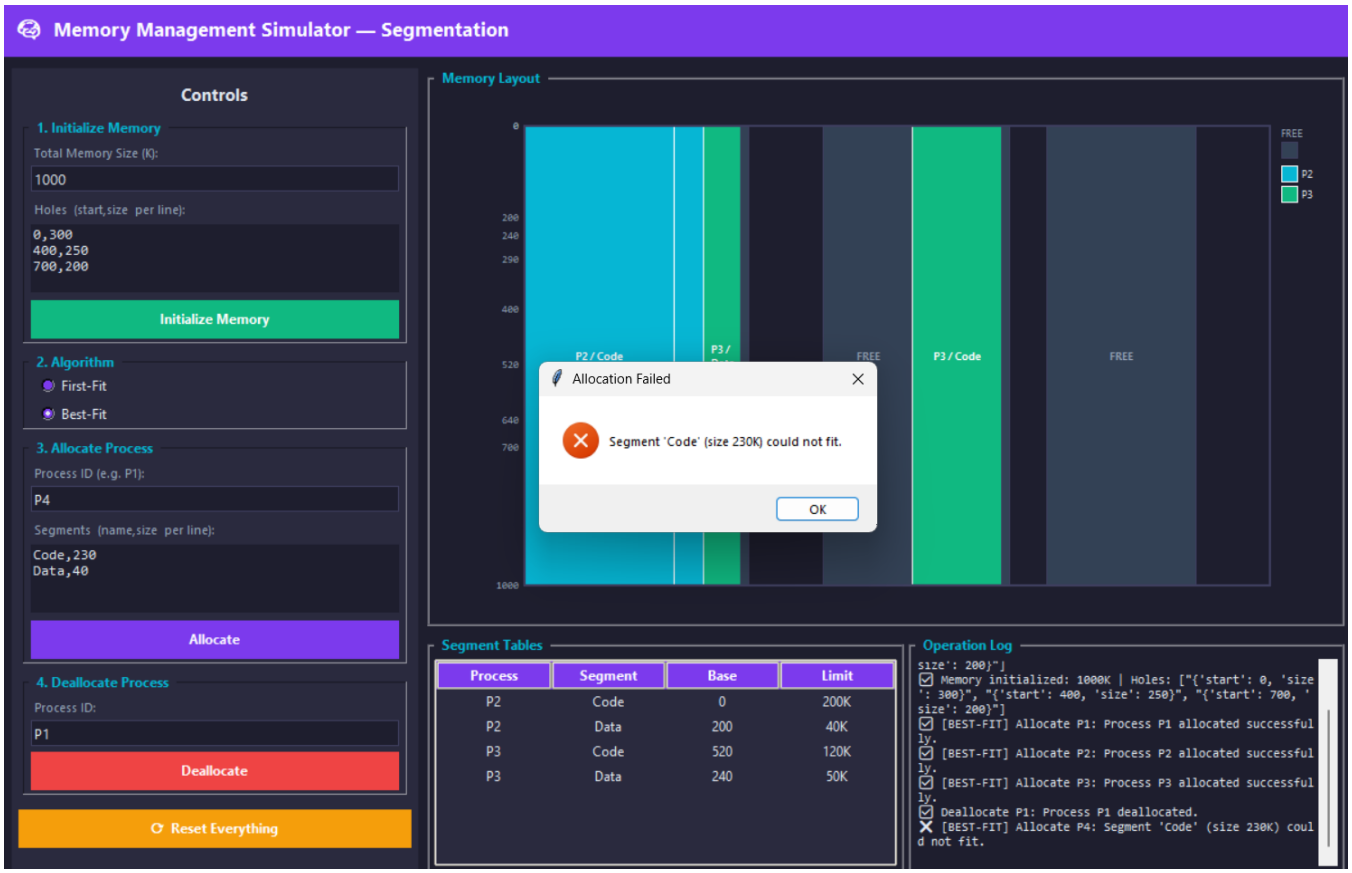
P1 segments Code@700/100K, Data@400/120K, Stack@800/90K are freed. Code(700) and Stack(800) are adjacent → merged into (700, 190K). Data(400) is adjacent to H2-rem at 520 → merged into (400, 120K). The three tiny 10K holes remain separate.



Snapshot 11 – After deallocating P1 (Best-Fit)

Step 5 – Allocate P4 (Code=230K, Data=40K) ← FAILS

Code (230K): the largest available hole is (700, 200K) — still 30K short. No hole fits Code, so P4 is rejected. The intense fragmentation left by Best-Fit's earlier allocations means even after P1's deallocation the memory is too fragmented for P4.



Snapshot 12 – P4 rejected, memory unchanged (Best-Fit)

6. Comparison: First-Fit vs Best-Fit

The same scenario produces different outcomes depending on which algorithm is used:

Criterion	First-Fit	Best-Fit
P3 Allocation	❌ Fails (40K holes too small after P2)	✅ Succeeds (holes better conserved)
P4 Allocation	✅ Succeeds (230K hole available)	❌ Fails (largest hole only 200K)
Speed	Faster — stops at first match	Slower — scans all holes
Fragmentation	More external fragmentation	Less per-step, but tiny holes accumulate
Memory usage	Leaves larger unused tails	Minimises leftover per allocation

7. Conclusion

This project successfully implements a fully interactive Memory Management Simulator using Python and Tkinter. The simulator demonstrates the practical trade-offs between First-Fit and Best-Fit segmentation strategies through a real-time graphical interface and this detailed report.

Key observations from the test scenario:

- First-Fit is faster (O(n) early exit) and tends to keep large contiguous holes available in the upper address space, which allowed P4 (Code=230K) to be allocated after P1's deallocation.
- Best-Fit minimises wasted space per allocation, enabling P3 to fit where First-Fit could not. However, it fragmented the freed space into many small holes, preventing P4 from finding a 230K contiguous region.
- Neither algorithm is universally superior — the best choice depends on the workload and access patterns.

All assignment requirements have been met: interactive GUI, both allocation algorithms, graphical memory layout, per-process segment tables, hole coalescing on deallocation, and rejection messages when a process cannot fit.